
基于 ModernTCN 的 AGV 工况感知与坡度调度软件 V1.0 软件设计说明书

软件全称：基于 ModernTCN 的 AGV 工况感知与坡度调度软件

软件简称：ModernTCN-AGV 感知调度软件

版本号：V1.0

生成日期：2026-05-25

编制说明：本文档根据当前仓库源码、数据契约、项目流程清单和默认部署配置生成，用于软件著作权登记辅助材料。申请人应在提交前确认软件名称、著作权人、开发完成日期、首次发表日期和权属证明。

修订记录

版本	日期	内容	责任人
V1.0	2026-05-25	根据当前项目生成软著申请 辅助设计说明书	[申请人填写]

目录

- 第 1 章 软件概述
- 第 2 章 运行环境
- 第 3 章 总体架构
- 第 4 章 数据契约与输入特征
- 第 5 章 ModernTCN 模型模块
- 第 6 章 训练与评估模块
- 第 7 章 ONNX 导出与一致性验证
- 第 8 章 MATLAB 在线推理模块
- 第 9 章 闭环验证接口
- 第 10 章 结果分析与报告模块
- 第 11 章 用户使用说明
- 第 12 章 技术特点
- 第 13 章 数据安全、第三方依赖与权属边界
- 第 14 章 版本说明
- 附录 A 主要源码文件清单
- 附录 B 输入输出文件清单
- 附录 C 默认配置表
- 附录 D 运行命令示例
- 附录 E 术语表

第 1 章 软件概述

1.1 软件名称、简称与版本

本软件全称为“基于 ModernTCN 的 AGV 工况感知与坡度调度软件”，简称为“ModernTCN-AGV 感知调度软件”，版本号为“V1.0”。本说明书、源码页眉、申请字段草稿和一致性检查清单均以该名称和版本为统一标识。

软件名称中的“ModernTCN”指本软件采用的核心时序感知模型架构；“AGV 工况感知”指主工况分类和转向方向分类功能；“坡度调度”指坡度回归量输出及其面向 LPV-MPC 控制器的调度应用。

1.2 开发背景

本软件面向对角双转向驱动 AGV 的工况感知与坡度调度需求。AGV 在直行、转向、坡道、低速堵转和扰动条件下的传感量具有时序相关性，单点规则或固定参数控制难以完整描述实际工况变化。

项目通过 Python/PyTorch 实现 ModernTCN 多任务时序感知模型，通过统一数据契约组织训练数据，通过 ONNX 格式实现跨 Python/MATLAB 部署，并在 MATLAB/Simulink 环境中提供在线推理和闭环验证接口。AGV 车辆模型、LPV-MPC 控制器和 Simulink 闭环模型在本软件中作为验证平台和接口环境。

1.3 软件目标

本软件目标是提供一套面向 AGV 场景的 ModernTCN 多任务时序工况感知与坡度调度工具链。软件能够：

1. 使用 128 步、19 维输入窗口构建统一时序数据集；
2. 训练 ModernTCN 多任务模型，输出主工况分类、转向方向分类和坡度回归；
3. 将 PyTorch 模型导出为 ONNX 格式；
4. 在 Python 端和 MATLAB 端分别进行一致性验证；
5. 在 MATLAB 中加载 ONNX 模型，维护在线滑动窗口，完成逐步推理；
6. 通过 Simulink 接口封装将感知输出接入闭环验证平台。

1.4 应用对象与使用场景

软件适用于 AGV 控制算法研究、工况感知模型验证、坡度调度策略仿真和闭环性能验证。典型用户包括控制算法开发人员、仿真平台维护人员和研究型项目成员。

当前版本主要用于桌面仿真和算法验证。ONNXRuntime 核心推理链路可用于评估计算余量；MATLAB/Simulink 封装主要用于闭环仿真验证和调试。

1.5 软件边界

本申请包的软件主体是 ModernTCN 多任务时序感知模型、数据读取与窗口组织、训练与评估、ONNX 导出、ONNXRuntime 一致性验证、MATLAB 在线预测、Simulink 接口封装、坡度调度量输出与闭环验证接口。

以下内容在本申请包中仅作为运行支撑模块、验证环境或接口环境出现，不作为软件主体：

-
7. AGV 非线性车辆动力学完整平台（state_eq.m、output_eq.m 等）；
 8. LPV 线性化与 MPC 控制器完整平台（lin_agv_at_point.m、mpc_setup_single_interp.m 等）；
 9. Simulink 闭环控制模型本体（.slx 文件）；
 10. GRU/TCN 对照算法（src/gru/、src/TCN/）；
 11. 论文图表生成系统。

1.6 与 GRU 申请包的边界区别

ModernTCN 申请包以 Python 实现的 ModernTCN 网络、ONNX 导出和跨环境部署链路为主体；GRU 申请包以 MATLAB 实现的 GRU 训练、推理、状态分类器和 Simulink 封装为主体。两者可共用数据契约和验证平台，但源码鉴别材料、说明书章节主线、申请表功能描述必须分别突出各自算法和实现流程。

第 2 章 运行环境

2.1 硬件环境

软件可在普通 PC 工作站上运行，建议使用多核 CPU 和 16GB 及以上内存。若进行 ModernTCN 训练，可选 NVIDIA GPU 以缩短训练时间；若仅执行已训练模型的在线推理和闭环验证，CPU 环境也可运行。

2.2 软件环境

软件环境包括 Windows 10/11 或兼容桌面操作系统、Python 3.8+、PyTorch、ONNXRuntime、MATLAB/Simulink，以及必要的 MATLAB 工具箱。

2.3 开发语言

软件主体源代码由 Python 和 MATLAB 编写。Python 部分包括 ModernTCN 模型定义、数据加载、训练指标、训练入口、ONNX 导出和 ONNXRuntime 一致性检查。MATLAB 部分包括默认配置、predictor 加载、在线窗口维护、状态分类器、Simulink 接口封装和闭环验证接口。

2.4 Python 依赖

Python 部分依赖以下第三方库，均作为运行环境，不作为本软件自有源代码：

- 12. PyTorch：深度学习框架，用于 ModernTCN 模型定义和训练；
- 13. ONNXRuntime：ONNX 模型推理引擎，用于一致性验证和实时性测试；
- 14. NumPy/SciPy：数值计算和数据加载；
- 15. onnx/onnxscript：ONNX 导出辅助。

2.5 MATLAB/Simulink 依赖

MATLAB 部分依赖以下工具箱或能力：

- 16. MATLAB 基础环境；
- 17. Simulink：闭环仿真平台；
- 18. Deep Learning Toolbox（可选）：ONNX 模型导入支持；
- 19. Statistics and Machine Learning Toolbox（可选）：数据处理辅助。

2.6 输入输出文件环境

核心输入包括统一数据集 `.mat` 文件、数据契约 `.json` 文件、已训练的 ONNX 模型文件和 PyTorch 参考输出 `.mat` 文件。当前主线数据集为

`data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat`。

核心输出包括训练指标 CSV、checkpoint `.pt` 文件、ONNX 模型文件、一致性检查报告、闭环仿真 `logout` 和结果分析报告。运行结果主要存放在 `results/` 目录。

第 3 章 总体架构

3.1 离线训练架构

离线训练阶段使用 Python/PyTorch 实现。数据加载模块从统一数据集读取 128 步、19 维输入窗口，按 run 级策略划分为训练集、验证集和测试集，并仅在训练集上拟合 scaler。ModernTCN 模型接受归一化后的窗口输入，输出三任务结果。训练过程记录损失函数和指标，保存 checkpoint。

3.2 在线部署架构

在线部署阶段使用 MATLAB 实现。MATLAB 加载已导出的 ONNX 模型和 scaler 参数，维护 128 步滑动窗口。每个控制周期从 AGV 输出的 y_{raw} 中提取 19 维特征，归一化后送入窗口，按需调用 ONNX predictor 获取预测结果。状态分类器对 logits 进行解码，输出工况标签、转向标签和 θ 调度量。

3.3 数据流



3.4 控制/调用流

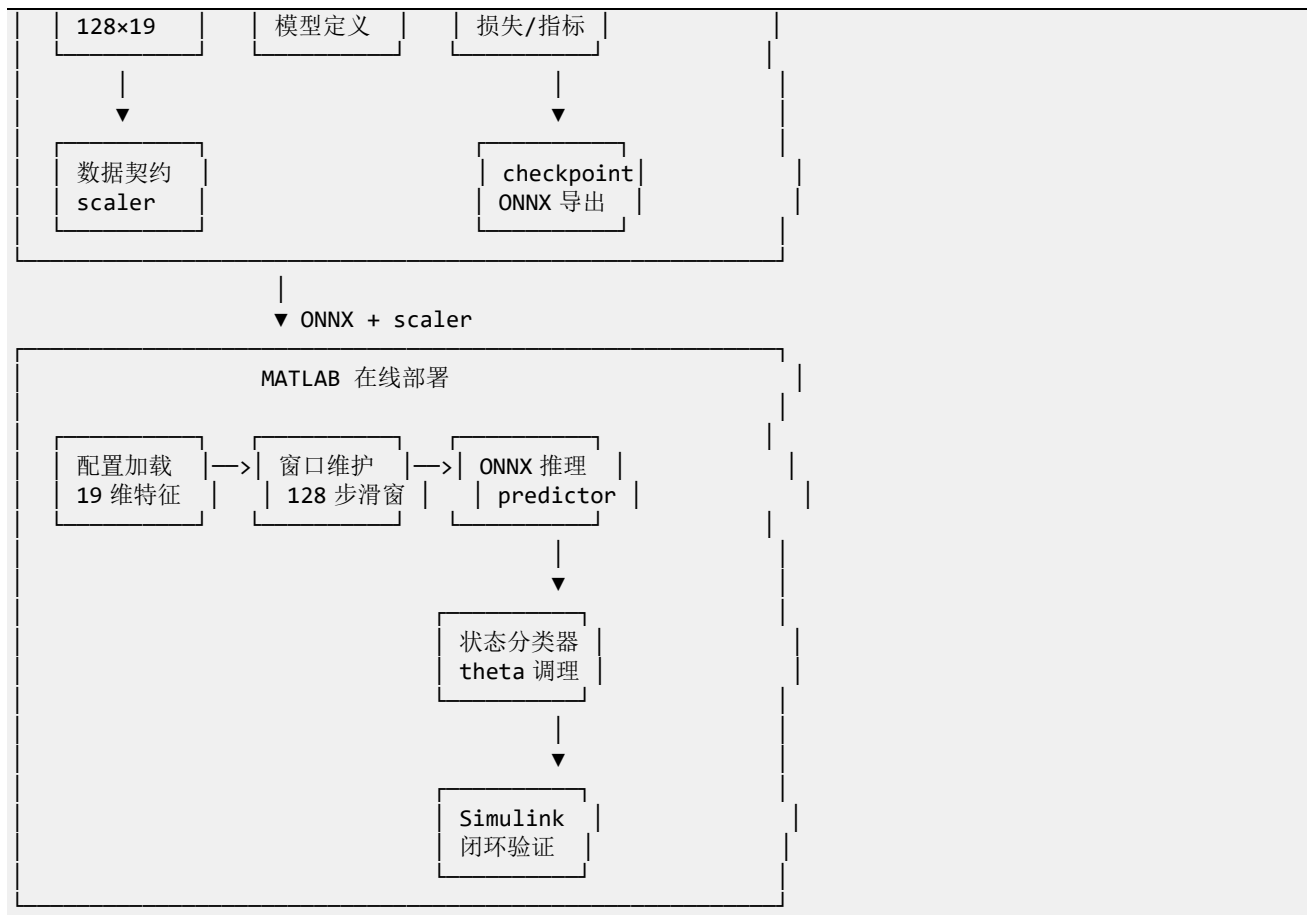
控制流由 Simulink 闭环模型驱动。每个仿真步中，AGV 模型输出传感和状态量，ModernTCN 在线模块维护历史窗口并完成预测，LPV-MPC 根据调度量更新控制器，控制输入再作用于车辆模型。

3.5 与 AGV-LPV-MPC 验证平台的接口关系

AGV 车辆模型、LPV 线性化、MPC 控制器和 Simulink 闭环模型在本软件中作为验证平台和接口环境。ModernTCN 感知输出通过 `preloadfcn_modern_tcn.m` 和 `ModernTCN_State_Classifier_sim.m` 接入 Simulink 模型，坡度调度量 θ_{hat} 用于 LPV-MPC 的 ρ 调度更新。

3.6 架构图





第 4 章 数据契约与输入特征

4.1 数据来源

训练数据由 AGV 仿真模型和参考路径共同生成。AGV 车辆对象为 `diagonal_dual_steer_drive_agv`，主动驱动/转向轮为 LF 和 RR，被动支撑轮为 RF 和 LR，控制/仿真采样周期为 $T_s = 0.01 \text{ s}$ 。

4.2 统一数据集

当前主线数据集为：

```
data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat
```

数据契约为：

```
data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2_contract.json
```

该数据集被 ModernTCN、GRU 和 TCN 共同使用，确保不同算法使用相同数据进行公平对比。

4.3 窗口化策略

窗口化策略固定为 `seq_len = 128`，控制采样周期为 $T_s = 0.01 \text{ s}$ ，因此单个输入窗口覆盖约 1.28 秒历史观测。窗口标签采用 `current_window_end` 策略，即以当前窗口末端对应工况作为标签。预测步长 `horizon_steps = 0`，当前默认版本不做未来步预测。

4.4 run 级划分与防泄漏策略

数据划分策略为 `run_level_no_window_leakage`。该策略在 run 级别（即每次独立仿真实验）划分训练集、验证集和测试集，避免同一 run 的不同窗口出现在不同集合中导致数据泄漏。

4.5 scaler 拟合策略

scaler 策略为 `fit_train_only_apply_val_test_online`。该策略仅在训练集上拟合均值和标准差，验证集、测试集和在线数据使用训练集的 scaler 参数进行归一化，避免信息泄漏。

4.6 19 维输入特征

当前 ModernTCN 输入维度为 `input_dim = 19`。19 个输入特征依次为：

序号	特征名	含义
1	accel_x	纵向加速度
2	gyro_z	偏航角速度
3	I_lf	左前轮电流
4	I_rr	右后轮电流
5	omega_wheel_lf	左前轮角速度
6	omega_wheel_rr	右后轮角速度
7	delta_lf	左前轮转向角
8	delta_rr	右后轮转向角
9	gyro_y	俯仰角速度

10	v_hat	估计速度
11	dv_hat_dt	速度变化率
12	ws_imbalance	轮速不平衡量
13	I_sum	电流和
14	I_diff_signed	有符号电流差
15	I_diff_abs	电流差绝对值
16	accel_x_lp	低通滤波纵向加速度
17	kappa_proxy	曲率代理量
18	accel_per_current	单位电流加速度
19	pitch_angle_est	估计俯仰角

4.7 三任务标签定义

本软件输出三类预测结果：

- 20. 主工况分类 (logits_main, 3 维)：flat=1, stall=2, slope=3，分别对应平地、堵转和坡道工况；
- 21. 转向方向分类 (logits_turn, 3 维)：right=-1, straight=0, left=1，分别对应右转、直行和左转；
- 22. 坡度/调度量回归 (theta_hat, 1 维)：连续值，表示估计的坡度角或调度量。

4.8 输入输出数据格式

训练阶段输入形状为 [batch, 128, 19]，进入模型前转置为 [batch, 19, 128] 以适配 Conv1d。ONNX 导出时固定输入名称为 input_window，形状为 [1, 128, 19]，输出名称为 logits_main、logits_turn 和 theta_hat。

第 5 章 ModernTCN 模型模块

5.1 模块定位

ModernTCN 模型模块是本软件的核心感知组件，负责从 128 步、19 维输入窗口中提取时序特征，完成主工况分类、转向方向分类和坡度回归三项任务。该模块定义在 `src/ModernTCN/modern_tcn_model.py` 中。

5.2 模型输入输出

模型输入为 `[batch, time=128, features=19]` 的归一化时序窗口。模型输出三个张量：

- 23. `logits_main`: `[batch, 3]`，主工况分类 logits；
- 24. `logits_turn`: `[batch, 3]`，转向方向分类 logits；
- 25. `theta_hat`: `[batch, 1]`，坡度/调度量回归输出。

5.3 ModernTCN-small 结构

ModernTCN-small 是本软件使用的默认模型结构，由以下组件组成：

- 26. `stem`: 1×1 卷积 + BatchNorm + ReLU，将 19 维输入映射到 64 通道；
- 27. `blocks`: 5 个 ModernTCNBlock 残差块，每块包含大核深度卷积和逐点 MLP；
- 28. `readout`: 从最后一个时间步、全局均值和全局最大值拼接特征，加上窗口级输入统计；
- 29. `任务头`: 三个独立的线性层分别输出主工况、转向方向和坡度。

默认配置参数：

```
input_dim = 19
seq_len = 128
channels = 64
blocks = 5
kernel_size = 31
temporal_padding = "same"
dropout = 0.15
expansion = 2
```

5.4 大核深度时序卷积

ModernTCNBlock 的核心是大核深度可分离卷积（depthwise convolution）。每个块中：

- 30. `depthwise`: `kernel_size=31` 的深度卷积，`groups=channels`，仅在时间维度上滑动；
- 31. BatchNorm + ReLU: 激活后接逐点卷积 pw1 扩展到 `hidden = channels × expansion` 维；
- 32. Dropout + pw2: 逐点卷积 pw2 压缩回 `channels` 维；
- 33. `layer_scale`: 可学习的缩放因子，初始值为 `1e-2`，用于稳定训练。

same padding 模式下 `padding = kernel_size // 2`，保证输出序列长度与输入一致。

5.5 残差连接与通道混合

每个 ModernTCNBlock 使用残差连接: $\text{output} = \text{residual} + y \times \text{layer_scale}$ 。残差连接保证梯度流畅传播, layer_scale 控制残差分支的贡献幅度。通道混合通过逐点卷积 (1×1 Conv) 实现, 在每个时间步上对通道维度进行线性变换。

5.6 多任务输出头

三个任务头共享卷积骨干提取的特征, 但使用独立的线性层:

- 34. **main_head**: `Linear(feature_dim, 3)`, 直接从拼接特征映射到 3 类主工况 logits;
- 35. **turn_head**: `Linear(turn_dim, 64) → ReLU → Linear(64, 3)`, 两层 MLP 映射到 3 类转向 logits;
- 36. **theta_head**: `Linear(feature_dim, 1)`, 从拼接特征映射到 1 维坡度回归值。

`turn_head` 的输入来源可通过 `turn_head_source` 配置, 默认为 "full" (使用全部特征)。

5.7 窗口统计特征融合

模型在 readout 阶段融合窗口级统计特征:

- 37. **时序特征**: 卷积骨干最后一个时间步 `h_last`、全局均值 `h_mean`、全局最大值 `h_max`;
- 38. **输入统计**: 原始输入窗口的 `last`、`mean`、`std`、`max`、`min` (共 $\text{input_dim} \times 5 = 95$ 维)。

两类特征拼接后送入任务头, 为模型提供多尺度时序信息和输入分布信息。

5.8 same padding 默认部署说明

V1.0 默认部署使用 `temporal_padding = "same"` 模式。该模式下卷积输出序列长度与输入一致, 便于 ONNX 导出和 MATLAB 导入。`same padding` 对应的 ONNX 算子主要由标准 `Conv1d`、`BatchNorm`、`ReLU`、`Linear` 等构成, 兼容性好。

5.9 causal padding 消融说明

代码中同时实现了 `temporal_padding = "causal"` 模式的 `CausalDepthwiseConv1d`, 通过左侧填充 + `Slice` 保证因果性。该模式可作为扩展验证能力, 在消融实验中评估因果卷积对感知精度的影响。V1.0 主申请材料以 `same padding` 的 seed 21 为主, `causal` 模型不作为默认部署模型。

第 6 章 训练与评估模块

6.1 单 seed 训练入口

单 seed 训练入口为 `src/ModernTCN/train_modern_tcn.py`。该脚本接受命令行参数，加载数据集，构建 ModernTCN 模型，执行训练循环，保存 checkpoint 和训练历史。

主要参数包括数据集路径、seed、学习率、批大小、训练轮数和输出目录。训练过程记录训练集和验证集的损失函数和指标。

6.2 多 seed 训练入口

多 seed 训练入口为 `src/ModernTCN/run_modern_tcn_theta10_v2_multiseed.py`。该脚本依次运行多个 seed 的训练，汇总各 seed 的指标，生成对比报告。多 seed 训练用于评估模型在不同随机初始化下的稳定性和泛化能力。

6.3 数据加载

数据加载模块 `src/ModernTCN/modern_tcn_data.py` 负责从 `.mat` 文件读取数据集，执行 run 级划分，构建 DataLoader。该模块同时检查数据契约中的 `seq_len`、`input_dim`、标签映射和 split 策略，确保数据格式与训练配置一致。

6.4 损失函数与指标

损失函数与指标模块 `src/ModernTCN/modern_tcn_metrics.py` 定义多任务损失函数，包括：

- 39. 主工况分类损失：交叉熵损失，支持类别权重和正负样本权重；
- 40. 转向方向分类损失：交叉熵损失，支持类别权重；
- 41. 坡度回归损失：MSE 或平滑 L1 损失，支持平坦区域加权和负坡度加权。

总损失为三项加权和，权重通过 `lambda_turn` 和 `lambda_theta` 等参数控制。评估指标包括准确率、F1 分数、MAE、RMSE 等。

6.5 checkpoint 保存

训练过程保存 checkpoint，包含模型状态字典、优化器状态、训练配置、数据契约和 scaler 参数。checkpoint 文件为 `.pt` 格式，可用于恢复训练或导出 ONNX。

6.6 summary 和 history 输出

训练完成后保存 summary CSV（各 seed 最终指标汇总）和 history CSV（训练过程损失和指标曲线数据）。这些文件用于生成训练报告和对比分析。

6.7 训练报告生成

多 seed 训练完成后自动生成训练报告，包括各 seed 的最佳验证指标、测试集指标、训练曲线和收敛情况。报告以 CSV 和文本格式保存到结果目录。

6.8 异常检查

训练脚本包含以下异常检查：

- 42. 数据集文件是否存在；
- 43. 数据契约中的参数是否与训练配置一致；
- 44. 训练过程中损失是否出现 NaN 或 Inf；
- 45. checkpoint 保存是否成功。

第 7 章 ONNX 导出与一致性验证

7.1 ONNX 导出目标

ONNX 导出模块 `src/ModernTCN/export_modern_tcn_onnx.py` 将训练得到的 PyTorch checkpoint 转换为 ONNX 格式，用于 MATLAB 端在线推理。导出前强制调用 `model.eval()` 关闭 dropout，固定输入形状 `[1, 128, 19]`，不启用 dynamic axes。

7.2 输入输出名称

ONNX 模型的输入输出名称固定为：

- 输入：input_window，形状 `[1, 128, 19]`，float32；
- 输出：logits_main `[1, 3]`、logits_turn `[1, 3]`、theta_hat `[1, 1]`，float32。

7.3 PyTorch 参考输出

导出 ONNX 时同时保存 PyTorch 参考输出 .mat 文件，包含测试集样本的输入 `X_sample` 和 PyTorch 推理结果 `logits_main_pytorch`、`logits_turn_pytorch`、`theta_hat_pytorch`。该文件用于后续一致性检查。

7.4 ONNXRuntime 一致性检查

`src/ModernTCN/check_onnxruntime_consistency.py` 加载 ONNX 模型和 PyTorch 参考输出，使用 ONNXRuntime 对相同样本进行推理，逐输出比较最大绝对误差和相对误差。一致性阈值默认为 `atol=1e-5`，`rtol=1e-4`。

7.5 MATLAB ONNX 一致性检查

`src/ModernTCN/ModernTCN_check_matlab_onnx.m` 在 MATLAB 端加载 ONNX 模型，读取 PyTorch 参考输出，逐窗口比较 MATLAB ONNX 推理结果与 PyTorch 输出的差异。该检查验证 ONNX 模型在 MATLAB 导入后的数值一致性。

7.6 部署文件冻结规则

默认部署模型为 seed 21，对应：

```
run_tag = modern_tcn_theta10_uniform_h0_v2_seed21
ONNX 文件 = results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.onnx
```

部署冻结后不再修改 ONNX 文件和对应的 scaler 参数。如需测试其他 checkpoint，可在调用脚本中传入覆盖参数。

7.7 自动生成层的处理边界

`src/ModernTCN/generated_layers/` 目录包含 MATLAB 导入 ONNX 后自动生成的兼容层代码。这些文件不作为本软件自有源程序主体，在源码鉴别材料中默认排除。若在说明书中引用，仅标注为“自动生成兼容层，不作为自有源程序主体”。

第 8 章 MATLAB 在线推理模块

8.1 默认配置加载

`src/ModernTCN/ModernTCN_default_config.m` 定义默认部署配置，包括 `seed`、`run_tag`、数据集路径、ONNX 文件路径和 `theta` 输出调理参数：

```
seed = 21
run_tag = 'modern_tcn_theta10_uniform_h0_v2_seed21'
theta_output_gain = 1.0
theta_abs_limit = deg2rad(12.0)
theta_rate_limit = deg2rad(5.0)
theta_mpc_deadzone = deg2rad(2.0)
```

8.2 predictor 加载

`src/ModernTCN/ModernTCN_load_predictor.m` 加载 ONNX 模型文件，返回 MATLAB 可调用的 `predictor` 对象。加载时检查 ONNX 文件是否存在，若不存在则报错提示。

8.3 在线窗口维护

`src/ModernTCN/ModernTCN_online_step.m` 是在线推理的核心封装。每个控制周期，该函数：

46. 从 `y_raw` 提取 19 维可观测特征；
47. 使用 `scaler` 参数进行归一化；
48. 维护 128 步滑动窗口（先进先出）；
49. 按需调用 ONNX `predictor` 完成推理。

函数使用 MATLAB `persistent` 变量保存在线状态，支持 `reset` 和 `seed` 切换。

8.4 单窗口预测

`src/ModernTCN/ModernTCN_predict_window.m` 接受已组织好的 `[128, 19]` 归一化窗口，调用 ONNX `predictor` 输出三任务预测结果。该函数用于在线推理和离线批量评估。

8.5 状态分类器

`src/ModernTCN/ModernTCN_state_classifier.m` 实现完整的在线工况识别状态机，功能包括：

50. 初始化（`'init'`）：加载配置、数据集、`scaler` 和 `predictor`，初始化滑动窗口；
51. 更新（`'update'`）：从单帧 `y_raw` 提取特征、归一化、更新窗口、推理、解码 `logits`。

状态分类器输出包括：

- `label_main`: 主工况标签（1=flat, 2=stall, 3=slope）；
 - `label_turn`: 转向方向标签（-1=right, 0=straight, 1=left）；
 - `theta_hat_rad`: 坡度调度量（弧度）；
-

-
- `conf_main`、`conf_turn`: 分类置信度;
 - `buffer_count`: 当前窗口填充数;
 - `ready`: 窗口是否已满。

8.6 theta 输出调理

状态分类器对 `theta_hat` 进行以下调理:

- 52. **增益调整:** `theta_hat = theta_hat × theta_output_gain`;
- 53. **绝对值限幅:** 限制在 `[-theta_abs_limit, +theta_abs_limit]` 范围内;
- 54. **变化率约束:** 相邻时刻变化量不超过 `theta_rate_limit`;
- 55. **死区处理:** 绝对值小于 `theta_mpc_deadzone` 时置零。

8.7 限幅、deadzone 与变化率约束

限幅防止 `theta` 输出超出合理坡度范围; `deadzone` 避免微小坡度估计引起 MPC 频繁调度; 变化率约束防止 `theta` 突变导致控制不平滑。这些调理措施保证感知输出适合闭环控制使用。

8.8 Simulink 包装函数

`src/ModernTCN/ModernTCN_State_Classifier_sim.m` 是 Simulink MATLAB Function 块的薄包装, 调用 `ModernTCN_online_step` 完成在线推理。该函数将 Simulink 信号转换为函数调用格式, 并将结果写入 `base workspace` 供后续模块读取。

第 9 章 闭环验证接口

9.1 验证平台说明

AGV 车辆模型、LPV 线性化、MPC 控制器和 Simulink 闭环模型在本软件中作为验证平台和接口环境。该平台用于检查 ModernTCN 感知输出在闭环控制场景中的可调用性和稳定性，不是本软件的主体功能。

9.2 preloadfcn_modern_tcn

`src/core/preloadfcn_modern_tcn.m` 是 Simulink 模型的预加载函数，负责初始化 ModernTCN 在线推理状态。该函数在 Simulink 模型加载时自动执行，调用 `ModernTCN_state_classifier('init', params)` 完成配置和 predictor 加载。

9.3 Simulink 模型接口

Simulink 模型通过 MATLAB Function 块调用 `ModernTCN_State_Classifier_sim.m`，在每个仿真步获取工况标签、转向标签和 θ 调度量。感知输出通过 Simulink 信号线传递给 LPV-MPC 调度模块。

9.4 与 LPV-MPC 调度量的接口

θ_{hat} 作为坡度调度量传递给 LPV-MPC 控制器，用于更新线性化网格点的权重和控制律。该接口通过 `mpc_update_from_rho.m` 实现， θ_{hat} 经过调理后作为 ρ 调度变量参与 MPC 在线更新。

9.5 输出日志与结果分析

闭环仿真完成后，`logstdout` 记录各时刻的工况标签、转向标签、 θ_{hat} 、控制输入和车辆状态。

`ModernTCN_analyze_closed_loop_out.m` 解析 `logstdout`，计算工况识别准确率、 θ 估计误差和轨迹跟踪指标。

9.6 闭环验证不是本软件主体的说明

本软件的核心功能是 ModernTCN 多任务时序感知模型的训练、导出、在线推理和接口封装。AGV 车辆模型、LPV-MPC 和 Simulink 闭环模型仅作为验证环境，用于检查感知输出的可用性。闭环验证平台的详细设计不属于本说明书的主线范围。

第 10 章 结果分析与报告模块

10.1 闭环输出分析

`src/ModernTCN/ModernTCN_analyze_closed_loop_out.m` 解析闭环仿真 `logout`，提取工况分类序列、转向分类序列和 `theta_hat` 时间序列，计算分类准确率、`theta` MAE/RMSE 和轨迹跟踪误差。

10.2 yraw 回放

`src/ModernTCN/ModernTCN_replay_closed_loop_yraw.m` 从保存的 `yraw` 日志中回放在线推理过程，用于调试和验证。回放模式不依赖 Simulink 模型，可在纯 MATLAB 环境中运行。

10.3 theta 散点图

`src/ModernTCN/plot_modern_tcn_theta_scatter.m` 绘制 `theta_hat` 与真实坡度的散点图，用于可视化评估坡度回归精度。

10.4 theta sweep 评估

`src/ModernTCN/eval_modern_tcn_theta_sweep_plot.m` 在不同坡度条件下的闭环仿真结果上评估 `theta` 估计性能，生成坡度 `sweep` 对比图。

10.5 实时性测试

`src/Compare/benchmark_modern_tcn_onnx_runtime.py` 测量 ONNXRuntime 单窗口推理延迟。

`src/Compare/run_realtime_benchmark.m` 在 MATLAB 端测量核心链路（ONNX 推理 + MPC 求解）的计算时间，评估相对于 10ms 控制周期的计算余量。

10.6 与 GRU/TCN 比较的边界

本软件提供 GRU 和 TCN 对照算法的闭环比较入口，但 GRU/TCN 训练和推理源码不属于本申请包的主体。比较结果仅用于验证 ModernTCN 感知输出的相对性能，GRU/TCN 的详细实现属于另一个独立的软著申请包。

第 11 章 用户使用说明

11.1 初始化项目

在 MATLAB 中切换到项目根目录后运行：

```
init_project;  
root = project_root();  
out_dir = results_dir();
```

11.2 检查数据集

当前主线数据集为：

```
data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat
```

用户应优先检查数据契约中的 `Ts=0.01`、`seq_len=128`、`input_dim=19`、标签映射、split 策略和 scaler 策略。

11.3 训练 ModernTCN

Python 单 seed 训练入口为：

```
python src/ModernTCN/train_modern_tcn.py --dataset-file  
data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat
```

11.4 多 seed 训练

多 seed 训练入口为：

```
python src/ModernTCN/run_modern_tcn_theta10_v2_multiseed.py
```

具体参数以脚本中的 `argparse` 定义为准。

11.5 导出 ONNX

默认部署模型位于：

```
results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.onnx
```

可使用以下入口导出：

```
python src/ModernTCN/export_modern_tcn_onnx.py --checkpoint  
results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.pt
```

11.6 检查 ONNXRuntime 一致性

```
python src/ModernTCN/check_onnxruntime_consistency.py
```

11.7 检查 MATLAB 一致性

在 MATLAB 中运行：

```
ModernTCN_check_matlab_onnx();
```

11.8 加载 MATLAB predictor

```
cfg = ModernTCN_default_config(project_root());  
[predictor, info] = ModernTCN_load_predictor(cfg);
```

11.9 运行 Simulink 闭环验证

```
init_project;  
cfg = ModernTCN_default_config(project_root());  
load_system('simulink/LPVMPC_AGV_simulink_Modern_TCN.slx');  
sim('simulink/LPVMPC_AGV_simulink_Modern_TCN.slx');
```

11.10 查看结果

主要结果输出到 **results/** 目录。ModernTCN 训练结果位于 **results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/**。

第 12 章 技术特点

12.1 多任务时序工况感知

本软件使用单个 ModernTCN 模型同时完成主工况分类、转向方向分类和坡度回归三项任务，共享卷积骨干特征，减少重复计算。

12.2 大核时序卷积结构

ModernTCN 采用 $\text{kernel_size}=31$ 的大核深度可分离卷积，配合残差连接和 layer_scale ，在保持参数量可控的同时捕获长时间依赖。

12.3 统一数据契约

本软件使用统一数据契约定义输入窗口长度、特征维度、标签映射、划分策略和 scaler 策略，确保训练、测试和在线部署遵循相同的数据格式。

12.4 跨 Python/MATLAB 部署

本软件通过 ONNX 格式实现 PyTorch 模型到 MATLAB 的跨语言部署，提供 ONNXRuntime 和 MATLAB 两端的一致性验证，确保数值行为一致。

12.5 面向闭环调度的 θ 输出调理

本软件对 θ_{hat} 进行增益调整、绝对值限幅、变化率约束和死区处理，使感知输出适合 LPV-MPC 闭环控制使用。

12.6 可复现实验与验证

本软件提供多 seed 训练、ONNX 一致性检查、闭环验证和实时性测试，支持从离线训练到在线部署的全流程可复现验证。

第 13 章 数据安全、第三方依赖与权属边界

13.1 本地数据处理

本软件所有数据处理均在本地完成，不涉及网络传输或云端存储。训练数据、模型权重和仿真结果均保存在本地文件系统。

13.2 第三方依赖声明

以下第三方库作为运行环境或依赖工具，不作为本软件自有源代码：

- 56. PyTorch：深度学习框架；
- 57. ONNXRuntime：ONNX 推理引擎；
- 58. NumPy/SciPy：数值计算库；
- 59. MATLAB/Simulink：仿真平台；
- 60. onnx/onnxscript：ONNX 导出辅助。

13.3 模型权重与训练数据不作为源程序

`.pt`（PyTorch checkpoint）、`.onnx`（ONNX 模型）、`.mat`（数据和中间结果）文件属于模型权重、部署产物或数据文件，不作为源程序鉴别材料纳入。

13.4 自动生成代码处理

`src/ModernTCN/generated_layers/` 为 MATLAB 导入 ONNX 后自动生成的兼容层代码，默认不纳入本次核心源程序鉴别材料。若申请人决定纳入，应单独标注为自动生成兼容层。

13.5 权属确认事项

申请人应最终确认以下事项：

- 61. 是否存在职务开发、合作开发、委托开发或第三方开源代码；
 - 62. GitHub 公开仓库若构成首次发表，首次发表日期需由申请人确认；
 - 63. 仓库中是否包含他人开源代码或单位职务成果。
-

第 14 章 版本说明

14.1 V1.0 功能范围

V1.0 版本包含以下功能：

- 64. ModernTCN-small 多任务时序感知模型定义；
- 65. 统一数据集读取与窗口化；
- 66. 多任务损失函数与训练指标；
- 67. 单 seed 和多 seed 训练入口；
- 68. ONNX 模型导出；
- 69. ONNXRuntime 和 MATLAB 一致性验证；
- 70. MATLAB 在线窗口维护与预测；
- 71. 状态分类器与 theta 输出调理；
- 72. Simulink 接口封装；
- 73. 闭环验证接口与结果分析。

14.2 不包含的功能

V1.0 版本不包含以下功能：

- 74. 嵌入式控制器固件；
- 75. 实时操作系统部署；
- 76. 完整的 AGV 车辆动力学仿真平台（作为验证环境引用）；
- 77. 完整的 LPV-MPC 控制器平台（作为验证环境引用）；
- 78. GRU/TCN 对照算法的训练和推理（属于另一个申请包）；
- 79. 在线学习或模型自适应更新。

14.3 后续扩展方向

后续版本可能扩展以下方向：

- 80. causal padding 模式的正式部署支持；
 - 81. 多 AGV 场景的感知模型；
 - 82. 嵌入式平台的 ONNX Runtime 部署；
 - 83. 在线增量学习能力。
-

附录 A 主要源码文件清单

序号	文件路径	语言	功能
1	init_project.m	MATLAB	项目初始化
2	project_root.m	MATLAB	根目录定位
3	results_dir.m	MATLAB	输出目录规范
4	src/ModernTCN/modern_tcn_model.py	Python	ModernTCN 模型定义
5	src/ModernTCN/modern_tcn_data.py	Python	数据加载
6	src/ModernTCN/modern_tcn_metrics.py	Python	损失函数与指标
7	src/ModernTCN/train_modern_tcn.py	Python	单 seed 训练
8	src/ModernTCN/run_modern_tcn_theta10_v2_multiseed.py	Python	多 seed 训练
9	src/ModernTCN/export_modern_tcn_onnx.py	Python	ONNX 导出
10	src/ModernTCN/check_onnxruntime_consistency.py	Python	ONNXRuntime 一致性
11	src/ModernTCN/ModernTCN_default_config.m	MATLAB	默认配置
12	src/ModernTCN/ModernTCN_load_predictor.m	MATLAB	predictor 加载
13	src/ModernTCN/ModernTCN_predict_window.m	MATLAB	单窗口预测
14	src/ModernTCN/ModernTCN_online_step.m	MATLAB	在线逐步推理
15	src/ModernTCN/ModernTCN_state_classifier.m	MATLAB	状态分类器
16	src/ModernTCN/ModernTCN_State_Classifier_sim.m	MATLAB	Simulink 包装
17	src/ModernTCN/ModernTCN_check_matlab_onnx.m	MATLAB	MATLAB ONNX 一致性
18	src/core/preloadfcn_modern_tcn.m	MATLAB	闭环预加载入口
19	src/ModernTCN/ModernTCN_analyze_closed_loop_out.m	MATLAB	闭环输出分析
20	src/ModernTCN/ModernTCN_replay_closed_loop_yraw.m	MATLAB	yraw 回放
21	src/Compare/benchmark_modern_tcn_onnx_runtime.py	Python	实时性测试
22	src/Compare/run_realtime_benchmark.m	MATLAB	实时性汇总

附录 B 输入输出文件清单

类型	文件	说明
输入	data/tcn/ModernTCN_dataset_*.mat	统一数据集
输入	data/tcn/ModernTCN_dataset_*_contract.json	数据契约
输入	results/modern_tcn/*/modern_tcn_seed21.onnx	ONNX 模型
输入	results/modern_tcn/*/*_pytorch_reference.mat	PyTorch 参考输出
输出	results/modern_tcn/*/modern_tcn_seed21.pt	checkpoint
输出	results/modern_tcn/*/*.csv	训练指标
输出	results/modern_tcn/*/summary*.txt	训练报告

附录 C 默认配置表

参数	值	说明
seed	21	默认随机种子
seq_len	128	输入窗口长度
input_dim	19	输入特征维度

channels	64	卷积通道数
blocks	5	残差块数
kernel_size	31	卷积核大小
temporal_padding	same	时序填充模式
dropout	0.15	Dropout 比率
expansion	2	逐点卷积扩展比
theta_output_gain	1.0	theta 增益
theta_abs_limit	deg2rad(12)	theta 绝对值限幅
theta_rate_limit	deg2rad(5)	theta 变化率约束
theta_mpc_deadzone	deg2rad(2)	theta MPC 死区

附录 D 运行命令示例

```
# 单 seed 训练
python src/ModernTCN/train_modern_tcn.py --dataset-file
data/tcn/ModernTCN_dataset_agv_dualsteer_theta10_uniform_conf_h0_v2.mat

# 多 seed 训练
python src/ModernTCN/run_modern_tcn_theta10_v2_multiseed.py

# ONNX 导出
python src/ModernTCN/export_modern_tcn_onnx.py --checkpoint
results/modern_tcn/modern_tcn_theta10_uniform_h0_v2_seed21/modern_tcn_seed21.pt

# ONNXRuntime 一致性检查
python src/ModernTCN/check_onnxruntime_consistency.py
% MATLAB 初始化
init_project;
cfg = ModernTCN_default_config(project_root());

% MATLAB ONNX 一致性检查
ModernTCN_check_matlab_onnx();

% 加载 predictor
[predictor, info] = ModernTCN_load_predictor(cfg);

% Simulink 闭环验证
load_system('simulink/LPV MPC_AGV_simulink_Modern_TCN.slx');
sim('simulink/LPV MPC_AGV_simulink_Modern_TCN.slx');
```

附录 E 术语表

术语	含义
AGV	自动导引车 (Automated Guided Vehicle)
ModernTCN	现代时序卷积网络 (Modern Temporal Convolutional Network)
ONNX	开放神经网络交换格式 (Open Neural Network Exchange)
ONNXRuntime	ONNX 模型推理引擎
LPV	线性参数变化 (Linear Parameter-Varying)

MPC	模型预测控制 (Model Predictive Control)
logits	模型输出的未归一化分类分数
theta_hat	坡度/调度量回归输出
scaler	数据归一化参数 (均值和标准差)
checkpoint	训练过程保存的模型快照
depthwise convolution	深度可分离卷积
layer_scale	层缩放因子
run	一次独立仿真实验
seq_len	输入序列长度
input_dim	输入特征维度